



Fundusze  
Europejskie  
Wiedza Edukacja Rozwój



Rzeczpospolita  
Polska

Politechnika  
Warszawska

Unia Europejska  
Europejski Fundusz Społeczny



# Algorytmy w Inżynierii Danych

## Automatyczne różniczkowanie

Bartosz Chaber

# Automatyczne różniczkowanie - tryb w tył

Do tej pory pochodne propagowane były w przód (od zmiennych do ostatecznego wyniku funkcji). Oznacza to, że dla każdej zmiennej funkcji trzeba było wykonać osobny przebieg.

Co z sieciami neuronowymi? Funkcja, której chcemy policzyć gradient jest skalarna i zawiera dziesiątki, setki lub więcej parametrów, które chcemy optymalizować...

Łańcuch pochodnych można zacząć “zwijać” z drugiej strony (tzn. od strony wyniku).

Tryb “w przód”

$$\frac{df}{dx} = \frac{df}{dg} \cdot \left( \frac{dg}{dx} \cdot \frac{dx}{dx} \right)$$

$$\frac{df}{dx} = \left( \frac{df}{dg} \cdot \frac{dg}{dx} \right)$$

Tryb “w tył”

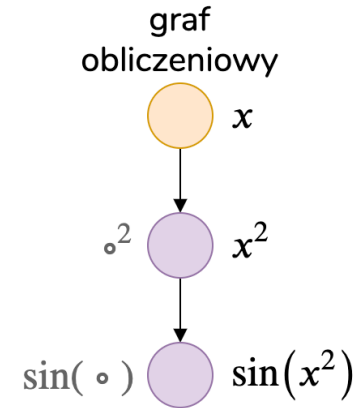
$$\frac{df}{dx} = \left( \frac{df}{df} \cdot \frac{df}{dg} \right) \cdot \frac{dg}{dx}$$

$$\frac{df}{dx} = \left( \frac{df}{dg} \cdot \frac{dg}{dx} \right)$$

# Budowanie grafu obliczeniowego

Zwykle algorytm automatycznego różniczkowania w trybie “w tył” wymaga stworzenia grafu obliczeniowego, który reprezentuje zależności między zmiennymi. Liście tego grafu to zmienne, a węzły to operacje.

Korzystając z mechanizmu przeciążania funkcji (w przypadku Julii – wielometod, *ang.* multiple dispatch) można łatwo zrealizować dynamiczne generowanie takiego grafu.

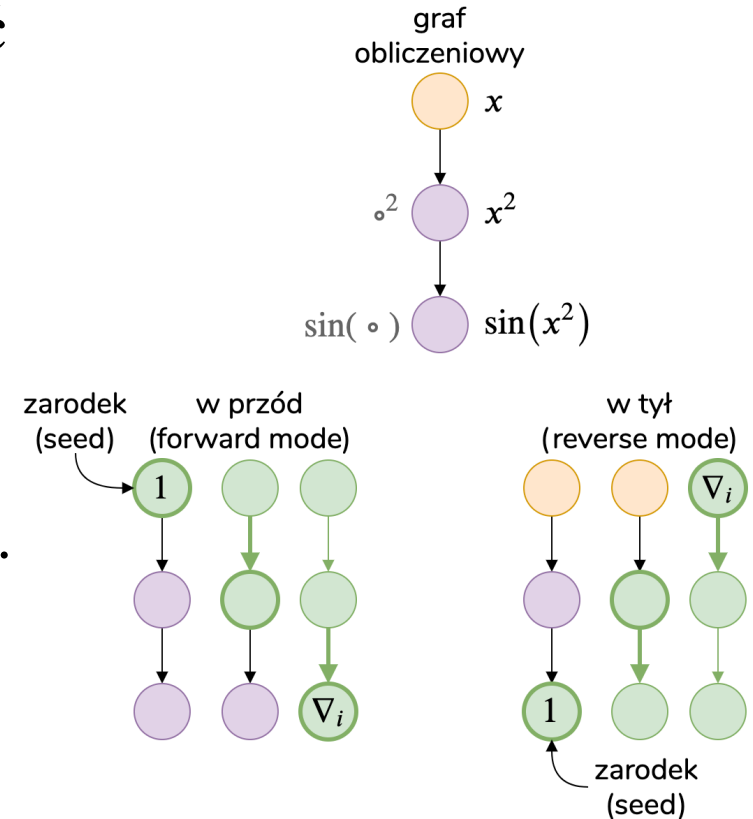


# Przebieg w przód, przebieg w tył

Aby obliczyć **wartość** funkcji, należy przejść graf w przód.

Aby efektywnie obliczyć **gradient** funkcji, należy przejść graf w tył.

Zarodek gradientu dodawany jest do **wynikowego** węzła grafu, nie do **zmiennej**.



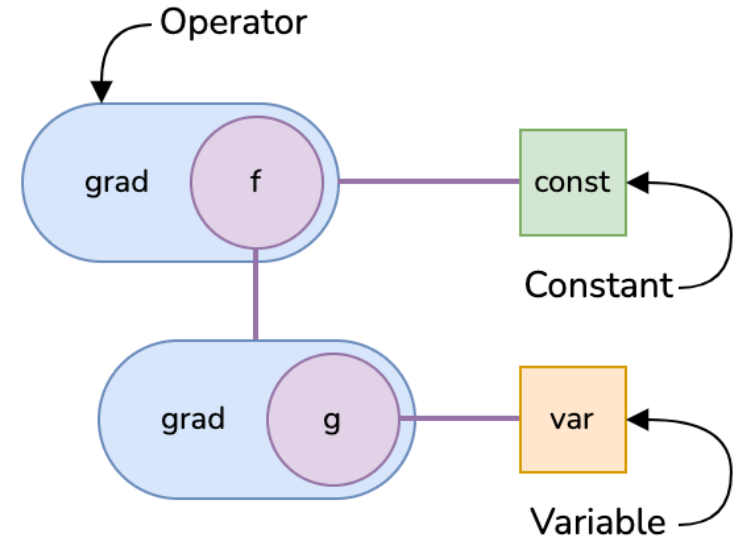
# Tryb w tył - przykładowa implementacja

Graf obliczeniowy złożony z trzech podstawowych rodzajów węzłów:

Constant, Variable i Operator:

- Constant posiada zdefiniowaną wartość, ale nie przechowuje informacji o gradiencie;
- Variable przechowuje informacje o swojej wartości oraz o gradiencie wyniku po “sobie”;
- Operator reprezentuje funkcję działającą na swoich argumentach (wyróżnimy potem operatory działające na wartościach skalarnych i na wektorach).

```
abstract type GraphNode end
abstract type Operator <: GraphNode
end
struct Constant{T} <: GraphNode
    output :: T
end
mutable struct Variable <: GraphNode
    output :: Any
    gradient :: Any
end
```



```
mutable struct ScalarOperator{F} <: Operator
  inputs :: Any
  output :: Any
  gradient :: Any
end
```

```
mutable struct BroadcastedOperator{F} <: Operator
  inputs :: Any
  output :: Any
  gradient :: Any
end
```



# Przykład budowania grafu obliczeniowego

Zdefiniujmy nasze zmienne:

```
# Variable(output) = new(output, nothing)
# ScalarOperator(fun, inputs...) = new{typeof(fun)}(inputs,
nothing, nothing)
```

```
x = Variable(5.0)
y = ScalarOperator(sin,
    ScalarOperator(*,
        x, x
    )
)
```

Przeciążając kolejne metody możemy na ich podstawie budować graf obliczeniowy:

```
import Base: +, -, *, /  
+(x::GraphNode, y::GraphNode) = ScalarOperator(+, x, y)  
-(x::GraphNode, y::GraphNode) = ScalarOperator(-, x, y)  
*(x::GraphNode, y::GraphNode) = ScalarOperator(*, x, y)  
/(x::GraphNode, y::GraphNode) = ScalarOperator(/, x, y)
```

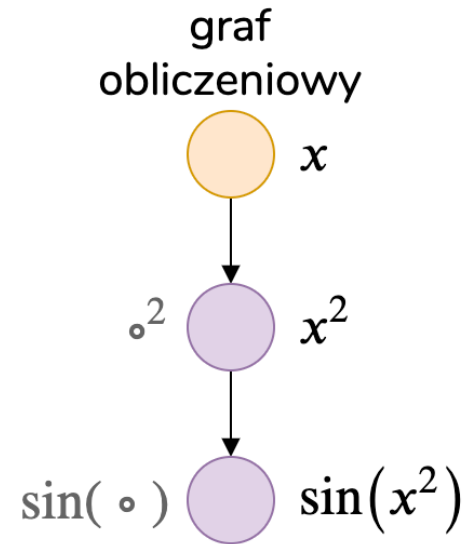
```
import Base: sin  
sin(x::GraphNode) = ScalarOperator(sin, x)
```

Ostatnie uzyskując przyjemną notację:

```
f(x) = sin(x*x)
```

```
x = Variable(5.0)
```

```
y = f(x)
```



# Budowanie grafu obliczeniowego

Aktualnie, mamy zbiór węzłów grafu, ale wyniki pośrednie nie posiadają wartości. Musimy znaleźć kolejność węzłów zapewniającą, że wyniki węzłów poprzednich są dostępne:

```
function topological_sort(head::GraphNode)
    visited = Set()
    order = Vector()
    visit(head, visited, order)
    return order
end
```

```
function visit(node,  
    visited, order)  
    if node ∉ visited  
        push!(visited, node)  
        push!(order, node)  
    end  
end
```

```
function visit(node::Operator,  
    visited, order)  
    if node ∉ visited  
        push!(visited, node)  
        for i in node.inputs  
            visit(i, visited, order)  
        end  
        push!(order, node)  
    end  
end
```

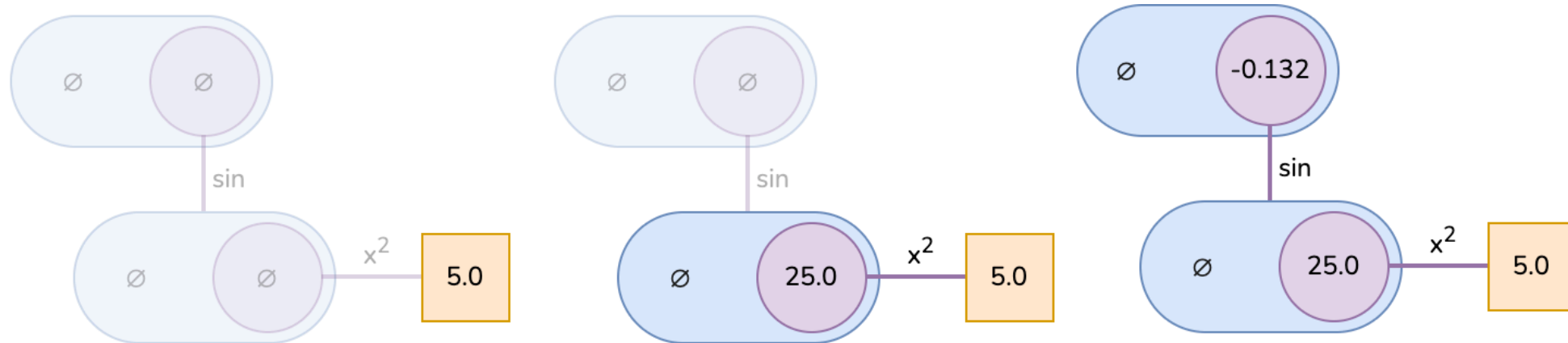
Aby obliczyć wartość węzła przejdziemy graf “w przód”, tj. od liści do korzenia i uaktualnimy wynik węzła przy użyciu funkcji forward resetując gradient wartością pustą.

```
reset!(node::Constant) = nothing
reset!(node::Variable) = node.gradient = nothing
reset!(node::Operator) = node.gradient = nothing

compute!(node::Constant) = nothing
compute!(node::Variable) = nothing
compute!(node::Operator) =
    node.output = forward(node,
        [input.output for input in node.inputs]...)
```

```
function forward!(order::Vector)
    for node in order
        compute!(node)
        reset!(node)
    end
    return last(order).output
end
```

## Budowanie grafu obliczeniowego





# Przechodzenie grafu w tył

W poniższym przykładzie, `x` jest uchwytem do węzła-liścia grafu, natomiast `y` jest uchwytem do węzła `Operator(sin)`, reprezentującego wynik funkcji.

```
x = Variable(5.0, 0.0)
y = sin(x*x)
```

Teraz, możemy rozpocząć propagację gradientu od węzła `y` poprzez `backward(y, seed=1.0)`.

```
update!(node::GraphNode, gradient) =  
if isnothing(node.gradient)  
    node.gradient = gradient  
else  
    node.gradient .+= gradient  
end  
function backward!(order::Vector; seed=1.0)  
    result = last(order)  
    result.gradient = seed  
    for node in reverse(order)  
        backward!(node)  
    end  
end
```

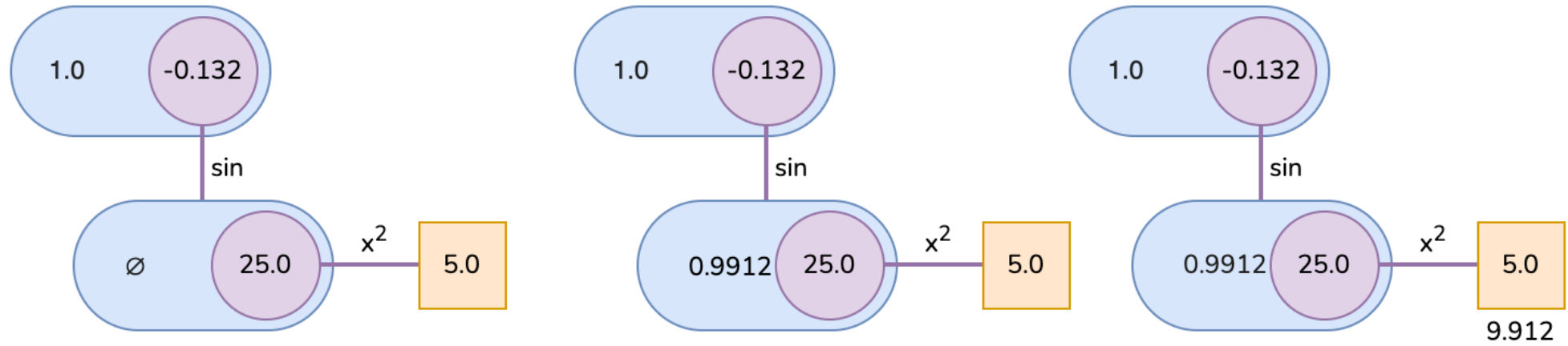
```
function backward!(node::Operator)
    inputs = node.inputs
    gradients = backward(node,
        [input.output for input in inputs]...,
        node.gradient)
    for (input, gradient) in zip(inputs, gradients)
        update!(input, gradient)
    end
end
```

# Przechodzenie grafu w tył

W poniższym przykładzie,  $x$  jest uchwytem do węzła-liścia grafu, natomiast  $y$  jest uchwytem do węzła `Operator(sin)`, reprezentującego wynik funkcji.

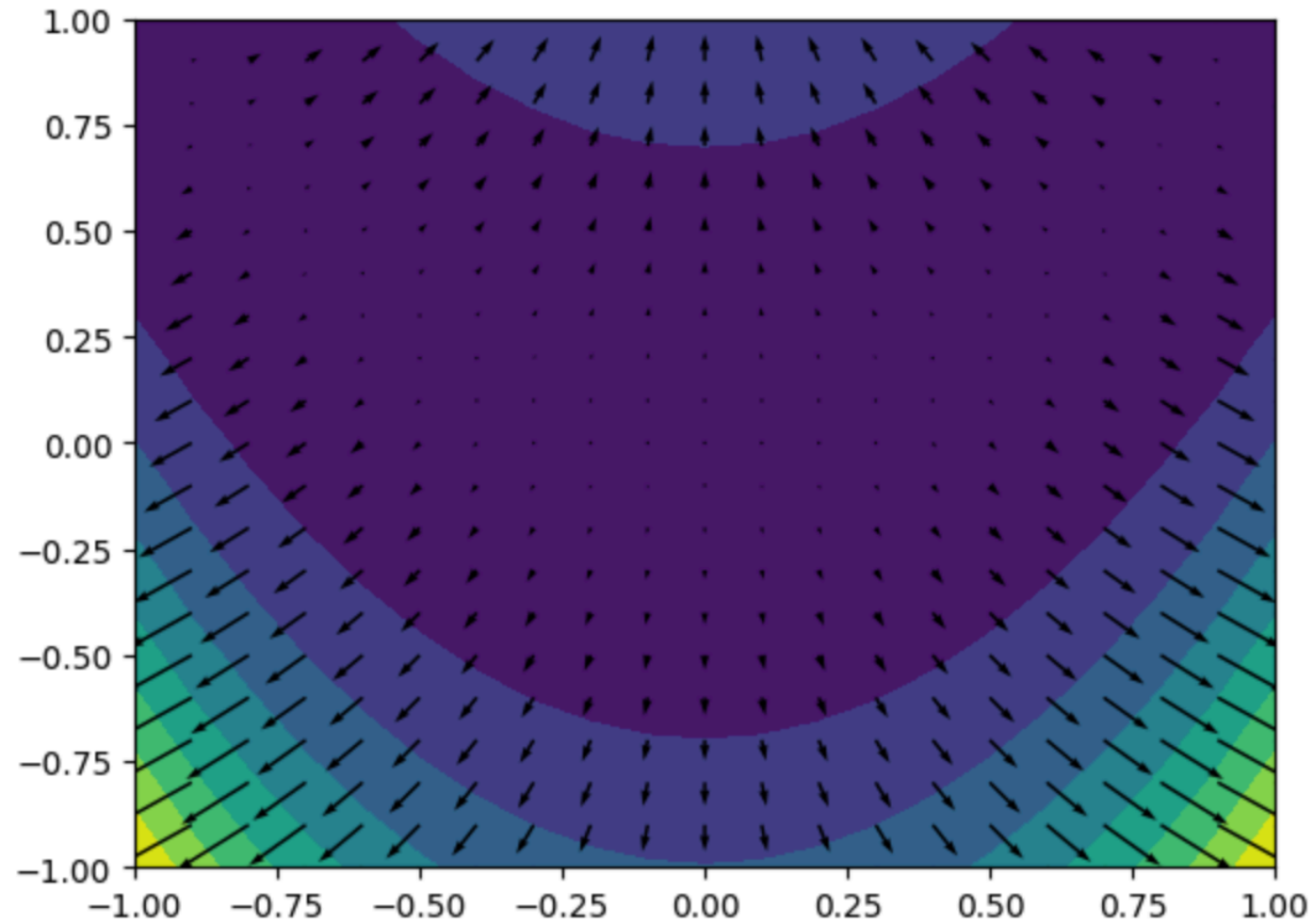
```
x = Variable(5.0, 0.0)
y = sin(x*x)
```

Teraz, możemy rozpocząć propagację gradientu od węzła  $y$  poprzez `backward(y, seed=1.0)`:



Ponownie policzymy gradient funkcji Rosenbrocka.

```
rosenbrock(x, y) =  
    (Constant(1.0) .- x .* x) .+  
    Constant(100.0) .* (y .- x .* x) .* (y .- x .* x)  
x = Variable([0.])  
y = Variable([0.])  
graph = topological_sort(rosenbrock(x, y))
```



```
v = -1:.1:+1
n = length(v)
z = zeros(n, n)
dzdx = zeros(n, n)
dzdy = zeros(n, n)
for i=1:n, j=1:n
    x.output .= v[i]
    y.output .= v[j]
    z[i,j] = first(forward!(graph))
    backward!(graph)
    dzdx[i,j] = first(x.gradient)
    dzdy[i,j] = first(y.gradient)
end
```



W przypadku różniczkowania funkcji wektorowych, mało efektywne jest konstruowanie macierzy pochodnych operacji, ponieważ interesuje nas “zwinięta” pochodna wynikowej funkcji skalarnej (czyli jej macierz pochodnych składa się tylko z jednej kolumny).

Wynika z tego, że podczas przejścia w tył potrzebujemy wyliczać tzw. iloczyn jakobianu i wektora, VJP – ang. Vector-Jacobian Product.

W zależności od tego, czy nasz wynik ma być wektorem wierszowym czy kolumnowym możemy to policzyć jako:  $\mathbf{r} \cdot \mathbf{J}$  lub  $\mathbf{J}^T \cdot \mathbf{c}$

Zarodek gradientu propagowany jest w **przeciwnym** kierunku, niż jest obliczane wyrażenie; Gradient propagowany jest do **wszystkich** zmiennych (liści grafu); Jest to wydajny sposób na różniczkowanie funkcji

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m, n > m$$

Zaprezentowane rozwiązania wykorzystują  
**przeciążanie** metod. W ten sposób  
wspierają **ograniczony** zbiór kodu w Julii  
Innym podejściem jest analiza **kodu** i  
**wygenerowanie** kodu różniczkującego.

# Literatura

- Roger Grosse, CSC321 Lecture 10: Automatic Differentiation, [https://www.cs.toronto.edu/~rgrosse/courses/csc321\\_2018/slides/lec10.pdf](https://www.cs.toronto.edu/~rgrosse/courses/csc321_2018/slides/lec10.pdf), dostęp: 11.03.2024;
- Atılım Güneş Baydin, Automatic Differentiation (or Differentiable Programming), <https://gbaydin.github.io/assets/pdf/slides-baydin-ad-atipp16.pdf>, dostęp: 11.03.2024.